



Lab 7

SE3082 – Parallel Computing

3rd Year 1st Semester – 2026

Name – Sooriyabandara D.M.L.P.

IT number – IT24101776

1. Comparison of MPI Programs

Program	MPI Operations Used	Memory Allocation	Manual Summation at Root	Result Availability
sum_bcast	MPI_Bcast, MPI_Send, MPI_Recv	Every process allocates the full array	Yes	Root only
sum_scatter	MPI_Scatter, MPI_Send, MPI_Recv	Root allocates the full array Other processes allocate only their chunk	Yes	Root only
sum_gather	MPI_Scatter, MPI_Gather	Root allocates the full array Other processes allocate only their chunk	Yes, root sums gathered values	Root only
sum_reduce	MPI_Scatter, MPI_Reduce	Root allocates the full array Other processes allocate only their chunk	No	Root only
sum_allreduce	MPI_Scatter, MPI_Allreduce	Root allocates the full array Other processes allocate only their chunk	No	All processes
sum_scan	MPI_Scatter, MPI_Scan	Root allocates the full array Other processes allocate only their chunk	No	Each process receives a different prefix sum

The expected sum for all programs is 500000500000.

2. Execution Time Comparison

The programs were executed using 2, 4, and 8 processes.

Program	2 Processes	4 Processes	8 Processes
Bcast	0.002567 sec	0.007300 sec	0.009900 sec
Scatter	0.002133 sec	0.001933 sec	0.002300 sec
Gather	0.002833 sec	0.003033 sec	0.000833 sec
Reduce	0.002567 sec	0.001833 sec	0.002567 sec
Allreduce	0.001967 sec	0.002967 sec	0.011833 sec
Scan	0.003861 sec	0.003730 sec	0.008776 sec

Results

- Fastest with 2 processes: MPI_Allreduce at 0.001967 seconds.
- Fastest with 4 processes: MPI_Reduce at 0.001833 seconds.
- Fastest with 8 processes: MPI_Gather at 0.000833 seconds.
- MPI_Bcast generally requires more communication because the complete array is sent to every process.
- MPI_Scatter is more memory-efficient because each process receives only the portion it needs.
- MPI_Reduce is efficient when only the root process needs the final result.
- MPI_Allreduce distributes the final result to every process, which may increase communication overhead.
- MPI_Scan calculates cumulative values and therefore performs a different operation from the other programs.

Execution times may vary depending on system load, processor availability and MPI implementation. Therefore, multiple runs and average values would provide more reliable results.

3. When to Use MPI_Scan Instead of MPI_Allreduce

MPI_Allreduce should be used when every process needs the same final result. For example, every process may need the total sum of an array.

MPI_Scan should be used when every process needs a cumulative result based on its rank. Each process receives the sum of its own value and the values contributed by all lower-ranked processes.

For example, if each process owns a number of records, MPI_Scan can calculate the global starting offset for each process: **sum_before_me = prefix_sum – local_sum**

This offset can be used to assign unique positions in a global output array. MPI_Scan is therefore useful for:

- Global array indexing
- Parallel prefix sums
- Cumulative distributions
- Histogram calculations
- Assigning output positions
- Load-balanced data distribution